Semantic News and Oil Price Database Project Report

May 9, 2026

Workspace: `DatabaseProject`
Primary stack: MySQL, pandas, scikit-learn, Plotly
Scope: Database design, data loading, feature engineering, forecasting, and reporting

Executive Summary

This project turns a collection of oil-market, geopolitical risk, semantic news, and country exposure CSV files into a reproducible analytics workflow. The codebase supports two complementary paths:

1. A MySQL-backed database layer with operational tables, dimensional tables, and reusable SQL views.
2. A time-series modeling layer that trains a transparent `StandardScaler -> Ridge` pipeline to predict next-trading-day Brent crude prices and produces a 10-day Monte Carlo forecast.

Validation in the local environment confirmed that the Python scripts compile, the model trains successfully, the forecast step runs end to end, and the visualization code builds the expected Plotly figures. The strongest one-day model result is a test RMSE of about **1.52 USD**, narrowly better than the previous-price baseline.

What makes the project useful is not just the model score, but the shape of the workflow. The database layer keeps the source data auditable and queryable, the modeling layer extracts a small but meaningful feature set from those tables, and the reporting layer packages the outputs into artifacts that are easy to present. That means the project is usable both as a demo and as a foundation for further iteration.

Project Goals

- Load the supplied CSV datasets into a MySQL database.
- Preserve the original operational tables while also creating BI-style dimension and fact tables.
- Provide SQL views that simplify analysis of oil prices, geopolitical risk, and event impact.
- Train and export a reproducible Brent price forecasting model.
- Generate forecast artifacts and visualizations that can be reused in the presentation deck and notebook.

Data Foundation

The workspace includes 20 CSV files in `datasets/`, covering market prices, geopolitical risk, events, country impact, and warehouse-ready reporting tables.

Dataset	Rows	Purpose
`ops_market_daily.csv`	4,047	Daily Brent/WTI prices, market indicators, lags, volatility, and event fields
`ops_gpr_daily.csv`	15,078	Daily geopolitical risk and semantic news measures
`ops_gpr_monthly.csv`	1,515	Monthly geopolitical risk summary data
`ops_gpr_country_monthly.csv`	66,660	Country-month geopolitical risk coverage
`ops_events.csv`	55	Named geopolitical and market events
`ops_countries.csv`	18	Country dimension for exposure analysis
`ops_petrol_price_snapshots.csv`	28	Country petrol price snapshots
`dim_* / fact_*` tables	mixed	Warehouse-ready reporting layer

The primary modeling table, `ops_market_daily.csv`, spans **2010-02-17 through 2026-03-12**. That gives the project a long enough time horizon for both model training and time-series evaluation.

Database Design

The database is organized into two layers.

Operational Layer

The operational layer preserves the source-style structure of the datasets. It is designed for traceability and for model feature engineering.

Important tables:

- `ops_market_daily`
- `ops_gpr_daily`
- `ops_gpr_monthly`
- `ops_gpr_country_monthly`
- `ops_events`
- `ops_countries`
- `ops_country_impact`
- `ops_petrol_price_snapshots`

Important joins:

- `ops_market_daily.market_date = ops_gpr_daily.gpr_date`
- `ops_market_daily.market_date = ops_events.event_date`

- ops_market_daily.market_date = ops_crude_oil_daily.trade_date
- ops_countries.country_id = ops_country_impact.country_id
- ops_countries.country_id = ops_petrol_price_snapshots.country_id
- ops_countries.iso3 = ops_gpr_country_monthly.iso3

Dimensional Layer

The dimensional layer supports reporting and BI-style analysis.

Key dimensions:

- dim_date
- dim_country
- dim_event

Key facts:

- fact_market_daily
- fact_gpr_daily
- fact_gpr_monthly
- fact_country_impact
- fact_petrol_prices

This structure keeps the original data intact while making the reporting side cleaner and more reusable.

The practical benefit is that the same data can be viewed in two ways. Analysts can work against the operational tables when they want source fidelity and flexible joins, while report authors can use the dimension and fact tables when they want stable keys and a more familiar warehouse-style layout. That separation makes the workspace easier to reason about than a single flattened table dump.

SQL Views

The loader applies `sql/analytics_views.sql` after the CSV import. It creates three useful views:

View	Description
vw_daily_oil_news_features	Daily Brent/WTI, GPR, event, and volatility features for analysis and modeling
vw_event_price_reaction	Event-date price and risk indicators for event-study style analysis
vw_country_petrol_impact	Country exposure, petrol price snapshots, and vulnerability indicators

Example query:

```
SELECT market_date, brent_price_usd, gpr_index, event_type, event_severity
FROM vw_daily_oil_news_features
WHERE event_flag = 1
ORDER BY market_date DESC
LIMIT 20;
```

Modeling Approach

The predictive target is the **next trading-day Brent crude oil price in USD**.

The training script, `src/train_oil_model.py`, uses:

- A chronological train/test split
- Feature scaling with `StandardScaler`
- `Ridge` regression for a transparent and reproducible baseline

That choice is appropriate for this project because it keeps the model easy to explain in a database-and-analytics setting, while still allowing a meaningful comparison against a naive previous-price baseline.

This is intentionally not a black-box forecasting stack. The goal is to keep the signal interpretable: price history, lags, volatility, a broad risk index, and a small set of event features. In a project review, that is often more defensible than a more complex model that is harder to trace back to the underlying business question.

Model Features

The model uses 20 core input features from `ops_market_daily.csv`:

Group	Columns
Prices	`brent_price_usd`, `wti_price_usd`
Market indicators	`dxy_index`, `vix_index`
Risk signal	`gpr_index`
Returns	`brent_return`, `wti_return`
Lags	`brent_lag_1`, `brent_lag_3`, `brent_lag_7`, `wti_lag_1`, `wti_lag_3`, `wti_lag_7`
Volatility	`brent_volatility_7d`, `brent_volatility_30d`, `wti_volatility_7d`, `wti_volatility_30d`
Spread	`brent_wti_spread`
Events	`event_severity`, `event_flag`

Two small derived features are also computed in the training and prediction scripts:

- 7-day Brent momentum
- Short-term acceleration
- Volatility regime ratio

Validated Results

The current project artifacts report the following h=1 results:

The improvement over the baseline is small, but that is a realistic outcome for next-day oil pricing. Yesterday's price is already a very strong predictor, so the real value of the model is the reproducible pipeline and the ability to incorporate market and risk context in a controlled way.

The train/test split is chronological, which is important here because the data is a time series rather than an exchangeable sample. That means the model is evaluated the way it would actually

Metric	Training	Test	Previous-price baseline
MAE	1.080	1.118	1.113
RMSE	1.567	1.516	1.536
MAPE	1.515%	1.463%	1.457%
R-squared	0.996	0.967	0.966

be used: trained on the past and judged on the future. The reported metrics therefore reflect a more realistic deployment-style check than a shuffled split would.

Horizon Results

The training script also fits direct multi-step models for horizons h=1 through h=10. In the local validation run, the longer-horizon models remained usable but naturally degraded as the forecast horizon increased. The h=10 model still achieved a meaningful fit, which supports the forecast fan produced by the Monte Carlo step.

Forecasting

`src/predict_oil_price.py` loads the trained h=1 model and generates a 10-day forward forecast.

Forecasting behavior:

- Runs Monte Carlo simulation paths
- Injects Gaussian noise calibrated to the h=1 test RMSE
- Applies a momentum blend using a recent linear trend
- Exports a `forward_forecast.csv` file with median, P10, P25, P75, and P90 values

That makes the output more informative than a single point estimate and gives the visualization layer a proper forecast fan.

Visualization

`Visualization/visualize_predictions.py` generates two interactive Plotly figures:

1. Actual vs predicted test-set scatter
2. Model vs baseline vs actual time series with the Monte Carlo forecast fan appended

The visualization step validated successfully in the local environment and produced both figures without error.

Those charts are doing a useful job in the story of the project. The scatter plot shows whether the model tracks actual price levels at all, while the time-series plot shows how the model compares with the baseline and how uncertainty expands across the 10-day forecast window. That makes the outputs easier to interpret than a table of metrics alone.

Validation Summary

The project was validated locally with the following checks:

- Python bytecode compilation passed for all core scripts.
- Required dependencies installed successfully in a temporary virtual environment.
- Model training ran end to end on the checked-in data.
- Monte Carlo forecasting completed successfully.
- Plotly visualizations built successfully.
- The core required feature columns are present in `ops_market_daily.csv`.

Notes and Risks

Two implementation details are worth noting:

- `main.py` currently relativizes artifact paths against the current working directory, which means it is not fully safe to launch from an arbitrary directory even though the file header suggests otherwise.
- `main_with_db.py` uses plain `INSERT INTO` behavior during MySQL loading, so rerunning the loader against an already populated database volume will likely trigger duplicate-key errors unless `-replace` behavior is added or conflict handling is introduced.

These are manageable issues, but they should be addressed before presenting the project as fully turnkey.

Two additional practical notes are worth keeping in mind. First, the forecast artifact is written alongside the trained model, which keeps the handoff simple, but it also means the artifact directory becomes the canonical output location for both training and prediction. Second, the validation run confirmed that the core feature columns are present in the dataset, so the current pipeline is not depending on fragile ad hoc preprocessing to succeed.

Conclusion

This project is a solid end-to-end example of a database-driven analytics workflow. It preserves the source data in a structured MySQL schema, adds analysis-ready SQL views, trains a transparent predictive model, and exports forecast artifacts that can be visualized and presented. The forecasting gain over baseline is modest, but the architecture is strong: it is reproducible, inspectable, and ready for iterative improvement.